

lab5

要求：在原先os-part4仓库中完成内存相关调用 `test_mem_lazy_allocation`
`test_mem_cow`

▼ 前置操作

- 完全不需要再git clone新的仓库，我们直接在os-part4这个目录下，运行这个指令

```
git fetch origin
git checkout part5-mem
```

切换到part5-mem分支下，再运行 `git checkout -b part5-lazy` 创建一个新的分支

▼ mem_lazy_allocation

- 先看一下要求：

```
#include "test.h"

#define SIZE (1 << 14) // 16KB
#define PAGES (SIZE / 4096) // 4KB per page

/*
 * Desc:
 * We test lazy allocation by allocating a large memory region but only
 * actually using a portion of it. We then check the physical page count
 * to ensure only the used pages are allocated.
 *
 * Expected:
 * Initially, no extra pages should be allocated. After accessing the first
 * few pages, only those pages should be allocated. The judge program will
 * check the physical page count at each step to verify
```

```

y lazy allocation.
*/

int main() {
    printf("Testing Lazy Allocation\n");

    // Get initial physical page count
    int initial_pages = getpgcnt();
    printf("Initial physical pages: %d\n", initial_pages);

    // Allocate a large memory region (should not allocate physical pages yet)
    char *mem = sbrk(SIZE);
    if (mem == (char*)-1) {
        printf("sbrk failed\n");
        exit(1);
    }

    int after_alloc_pages = getpgcnt();
    printf("Physical pages after sbrk: %d (should be same as initial)\n", after_alloc_pages);

    if (after_alloc_pages != initial_pages) {
        printf("ERROR: Physical pages increased without access!\n");
        exit(1);
    }

    // Access first page (should trigger page fault and allocate one page)
    mem[0] = 'A';
    int after_first_access = getpgcnt();
    printf("Physical pages after first access: %d (should be +1)\n", after_first_access);

    if (after_first_access != initial_pages + 1) {
        printf("ERROR: Expected %d pages, got %d\n",

```

```

initial_pages + 1, after_first_access);
    exit(1);
}

// Access a page in the middle
mem[SIZE/2] = 'B';
int after_mid_access = getpgcnt();
printf("Physical pages after mid access: %d (should be +2)\n", after_mid_access);

if (after_mid_access != initial_pages + 2) {
    printf("ERROR: Expected %d pages, got %d\n",
initial_pages + 2, after_mid_access);
    exit(1);
}

// Access last page
mem[SIZE-1] = 'C';
int after_last_access = getpgcnt();
printf("Physical pages after last access: %d (should be +3)\n", after_last_access);

if (after_last_access != initial_pages + 3) {
    printf("ERROR: Expected %d pages, got %d\n",
initial_pages + 3, after_last_access);
    exit(1);
}

printf("Lazy Allocation Test Completed Successfully\n");
exit(0);
}

```

这个测试样例中，重复调用了 `getpgcnt()` 用来检测是否在 `sbrk` 要求分配时，系统只进行了 `sz` 的调整，而没有真正执行物理页的分配，只有在三次真正向 `mem` 数组写入时才会分配真正的物理页。而 `mem` 数组在起初，本应由 `sbrk` 分配一个16KB=4Page大小的虚拟空间，但因为是懒分配，这些虚拟地址实际上并没有映射到真实的物理空间中。

- 在这个测试样例中，要实现懒分配，参照助教给出的指引，应该先实现 `getpgcnt` 用于统计全局已分配物理页数。运行 `make run_test TYPE=LAZY`，我们发现如下报错

```
pid 2 test_mem_lazy_a: unknown sys call 54325
pid 2 test_mem_lazy_a: unknown sys call 54325
pid 2 test_mem_lazy_a: unknown sys call 54325
```

查询 `sysnum.h`，发现 `#define SYS_getpgcnt 54325`，而 `syscall.c` 中并没有完成相应注册，所以我们添加这个 `sys_getpgcnt()` 的注册信息，然后在 `sysproc.c` 中进行完整的实现。

- 根据助教的提示，我们知道需要在 `kalloc.c` 中调整一些逻辑，用于统计全局的分配情况，而不是直接在当前进程中遍历页表。
 - ▼ 接下来要分析一下 `kalloc.c` 中的具体内容，看一下 xv6 是怎么内存管理的，以及我们该怎么下手操作。

```
// Physical memory allocator, for user processes,
// kernel stacks, page-table pages,
// and pipe buffers. Allocates whole 4096-byte
// pages.
```

```
#include "include/types.h"
#include "include/param.h"
#include "include/memlayout.h"
#include "include/riscv.h"
#include "include/spinlock.h"
#include "include/kalloc.h"
#include "include/string.h"
#include "include/printf.h"
```

```
void freerange(void *pa_start, void *pa_end);
```

```
extern char kernel_end[]; // first address after kernel.
```

```
struct run {
```

```

    struct run *next;
};

struct {
    struct spinlock lock;
    struct run *freelist;
    uint64 npage;
} kmem;

void
kinit()
{
    initlock(&kmem.lock, "kmem");
    kmem.freelist = 0;
    kmem.npage = 0;
    freerange(kernel_end, (void*)PHYSTOP);
#ifdef DEBUG
    printf("kernel_end: %p, phystop: %p\n", kernel_end, (void*)PHYSTOP);
    printf("kinit\n");
#endif
}

void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
        kfree(p);
}

// Free the page of physical memory pointed at
// by v,
// which normally should have been returned by
// a
// call to kalloc(). (The exception is when

```

```

// initializing the allocator; see kinit above.)
void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa <
kernel_end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    kmem.npage++;
    release(&kmem.lock);
}

// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r) {
        kmem.freelist = r->next;
        kmem.npage--;
    }
}

```

```

    release(&kmem.lock);

    if(r)
        memset((char*)r, 5, PGSIZE); // fill with j
unk
    return (void*)r;
}

uint64
freemem_amount(void)
{
    return kmem.npage << PGSHIFT;
}

```

- `extern char kernel_end[];`：这是一个外部定义的符号，由于物理内存分配器不能把内核自己正在用的内存分配出去。所以，可供分配的空闲物理内存，是从 `kernel_end` 开始，一直到物理内存的最高点 `PHYSTOP` 这里为 `0x80600000`。
- `struct run`：这是空闲链表 `freelist` 的节点结构体，它的前8Byte是指向下一个节点的指针，这样，每个空闲物理页的前8Byte就会自然地用来存放下一个空闲页的指针，由于这个页本身就是空闲的，这部分空间就被很好地利用了。**注意**，在这个 `kalloc.c` 中，传递的指针虽然都是 `(void*)`，但已经是**直接指向真实物理页地址**的指针了，因此这个整个脚本已经是虚拟内存页表与真实物理页的桥接。
- `struct kmem`：全局内存管理者，`freelist` 是全局空闲页链表的起始地址，`npage` 则是还有多少空闲页。
- `kinit()`：初始化 `kmem`，调用 `freerange`，将 `kernel_end[]` 起始到达物理内存最高点 `PHYSTOP` 处全部释放。
- `freerange(void *pa_start, void *pa_end)`：把给定范围内的物理内存，切成一个个 4096 字节（4KB，`PGSIZE`）的标准页，然后循环调用 `kfree` 把它们挂进空闲链表。
- `kfree(void *pa)`：
 - **垃圾填充（Catch dangling refs）**：`memset(pa, 1, PGSIZE)`。把要释放的这一整页全部填满 `1`。这是一种极佳的**防御性编程**。如果有恶意程序或写错的内核代码还在使用（Use-

after-free) 这块被释放的内存，读出来的全是一堆乱码（垃圾数据），很容易让程序立即崩溃报错，从而及早发现 Bug。

- **链表头插法：** 把这个物理页强转成 `struct run*`，然后采用头插法（LIFO，后进先出）挂入 `kmem.freelist`，最后 `npage++`。
- **`kalloc(void)`（这是几乎所有新分配/复制页表的核心函数）：**
 - **取走头部：** 加锁后，直接把 `kmem.freelist` 指向的第一个物理页拿出来（从链表头上摘除节点），然后 `npage--`。
 - **再次垃圾填充：** 如果分配成功，`memset((char*)r, 5, PGSIZE)` 把这页填满 5。原因同上，防止新分配的内存里残留着上一个进程的密码、密钥等机密数据，同时也防止未初始化就直接使用引发的随机 Bug。
- **`freemem_amount(void)`：** 把空闲页数 `npage` 左移 `PGSHIFT`（乘以 4096），得出空闲内存的总字节数。
- 之后我们发现这里可以直接用 `kmem` 的成员 `npage`，但是我们仍不知道总页数是多少，如果直接使用现有的 `freemem_amount() >> PGSHIFT`，则返回的是空闲页面数。因此在这里我们再声明一个全局变量 `total_pages` 并在 `init` 的时候对整个物理空间可分配的页面数做一个统计，补充一个接口函数，就大功告成。

```
uint64 total_pages = 0;

void freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char *)PGROUNDUP((uint64)pa_start);
    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
    {
        total_pages++;
        kfree(p);
    }
}

uint64
allocated_pages(void)
```



```

{
    acquire(&kmem.lock);
    uint64 pages = total_pages - kmem.npage;
    release(&kmem.lock);
    return pages;
}

```

记得在 `kalloc.h` 中声明这个新的函数，否则不能在 `sysproc.c` 中调用。
在 `sysproc.c` 中，我们添加函数定义：

```

uint64
sys_getpgcnt(void)
{
    return allocated_pages();
}

```

之后可以看到结果是这样的：

```

init: starting test_mem_lazy_allocation
testing output size:157, contents:
Testing Lazy Allocation
Initial physical pages: 40
Physical pages after sbrk: 44 (should be same as initial)
ERROR: Physical pages increased without access!
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test2 output:
Testing Lazy Allocation
Initial physical pages: 40
Physical pages after sbrk: 44 (should be same as initial)
ERROR: Physical pages increased without access!

```

在第一次 `sbrk` 的时候，直接分配了4个完整的物理页，所以与要求不符，但我们确实已经实现了 `sys_getpgcnt`。

- 下一步，我们继续调整 `sysproc.c` 中的 `sbrk` 逻辑，让它在被调用时只改变 `myproc()-sz`，而不真正分配物理页面。
 - `sys_sbrk` 的核心函数 `growproc()`，它在做的事情实际上也就是：调整 `sz`；在需要更多内存时调用 `uvmalloc`，在需要更少内存时调用 `uvmdealloc`。在这里我们就先不管少内存时的解映射。如果只是改变 `sz`，那我们直接把前面有关 `uvmalloc` 部分的调用去掉就好了，因为真正的分配过程要在用到时，触发缺页异常再来处理。

```
int growproc(int n)
{
    uint sz;
    struct proc *p = myproc();

    sz = p->sz;
    if (n > 0)
    {
        // 懒分配实现
        sz = sz + n;
    }
    else if (n < 0)
    {
        sz = uvmdealloc(p->pagetable, p->kpagetable, sz, sz + n);
    }
    p->sz = sz;
    return 0;
}
```

- 下一步，我们要处理 `sbrk` 中，涉及到解除映射的部分，也就是 `uvmdealloc` 的部分，这块先研究一下 `uvmalloc` 和 `uvmdealloc` 这两个函数：

▼ `uvmalloc`：

```
// Allocate PTEs and physical memory to grow process from oldsz to
// newsz, which need not be page aligned. Returns new size or 0 on error.
uint64
```

```

uvmalloc(pagetable_t pagetable, pagetable_t kpaget
able, uint64 oldsz, uint64 newsz)
{
    char *mem;
    uint64 a;

    if(newsz < oldsz)
        return oldsz;

    oldsz = PGROUNDUP(oldsz);
    for(a = oldsz; a < newsz; a += PGSIZE){
        // 每次循环前进一页 (4KB)。调 kalloc 要一块真实的物理内存。
        // 拿到后, 必须 memset 清零。
        mem = kalloc();
        if(mem == NULL){
            uvmdealloc(pagetable, kpagetable, a, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        // 第 1 次映射: 映射到用户页表
        if (mappages(pagetable, a, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R|PTE_U) != 0) {
            kfree(mem);
            uvmdealloc(pagetable, kpagetable, a, oldsz);
            return 0;
        }
        // 第 2 次映射: 映射到内核页表
        if (mappages(kpagetable, a, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R) != 0){
            int npages = (a - oldsz) / PGSIZE;
            vmunmap(pagetable, oldsz, npages + 1, 1);
            // plus the page allocated above.
            vmunmap(kpagetable, oldsz, npages, 0);
            return 0;
        }
    }
}

```

```
return newsz;
}
```

- 在 `kalloc` 这一页成功后，会进行两次 `mappages`：
 - **给用户用**：带有 `PTE_U` (User) 权限，这意味着 CPU 在用户态 (U Mode) 时可以读写执行它。
 - **给内核用**：没有 `PTE_U` 权限！为什么要去掉？因为 RISC-V 架构的安全机制规定，内核态 (S Mode) 默认情况下**绝对禁止**访问带有 `PTE_U` 标志的页面（除非设置 `sstatus.SUM` 位）。不带 `PTE_U`，内核页表把这个地址当成自己的特权内存，内核就可以直接用指针对其进行读写了。
- 而第二次映射到内核失败时，则会触发回滚机制
 - `vmunmap(pagetable, ..., 1)`：把用户页表的映射删掉。注意最后的参数 `1`，代表连带释放底层的物理内存 `kfree`。这里要清理 `npages + 1` 页（包含刚才刚映射进去的那一页）。
 - `vmunmap(kpagetable, ..., 0)`：把内核页表的映射删掉。注意最后的参数 `0`，代表只删页表项，不去二次 `kfree` 物理内存。

▼ `uvmdealloc`：

```
// Deallocate user pages to bring the process size
// from oldsz to
// newsz.  oldsz and newsz need not be page-aligne
// d, nor does newsz
// need to be less than oldsz.  oldsz can be large
// r than the actual
// process size.  Returns the new process size.
uint64
uvmdealloc(pagetable_t pagetable, pagetable_t kpag
etable, uint64 oldsz, uint64 newsz)
{
    if(newsz >= oldsz)
        return oldsz;
    // 注意到，如果newsz和oldsz在对齐后的内存大小相等，则仍不
    // 触发vmunmap的解除映射
    if(PGROUNDUP(newsz) < PGROUNDUP(oldsz)){
        int npages = (PGROUNDUP(oldsz) - PGROUNDUP(new
```

```

sz)) / PGSIZE;
    // 解除这些页面的内核映射和用户映射
    vmunmap(kpagetable, PGROUNDUP(newsz), npages,
0);
    vmunmap(pagetable, PGROUNDUP(newsz), npages,
1);
}

return newsz;
}

```

- 接下来，我们应该修改 `vmunmap` 的定义

```

void vmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if ((va % PGSIZE) != 0)
        panic("vmunmap: not aligned");

    for (a = va; a < va + npages * PGSIZE; a += PGSIZE)
    {
        if ((pte = walk(pagetable, a, 0)) == 0)
            // panic("vmunmap: walk");
            continue; // 懒分配修改 1: 如果没有分配过页表目录，直接跳过当前页

        if ((*pte & PTE_V) == 0)
            // panic("vmunmap: not mapped");
            continue; // 懒分配修改 2: 如果页表项存在但无效（没挂物理页），直接跳过当前页

        if (PTE_FLAGS(*pte) == PTE_V)
            panic("vmunmap: not a leaf");
    }
}

```

```

    if (do_free)
    {
        uint64 pa = PTE2PA(*pte);
        kfree((void *)pa);
    }
    *pte = 0;
}
}

```

这样，在解除空头支票的时候，不会在 `walk` 找不到对应物理页面时报错了。

- 下一步，我们需要处理 `trap.c` 中的中断处理过程，当产生缺页异常时，要给这个引发异常的地址真正分配一个物理页。新增 `lazy_handler` 处理函数，每次发生Page Fault时（`r_scause` 的值可以判断）调用。处理函数校验Page Fault的地址是否是**进程已分配地址空间内的合法地址**，如果是合法地址，使用 `kalloc` **分配页面并重新映射到页表**。

现在如果我们再次运行这个测试，则会出现这样的报错，这个报错的 `scause` 就是写缺页异常的错误号了。

```

usertrap(): unexpected scause 0x000000000000000f p
id=2 test_mem_lazy_a
sepc=0x00000000000000a4 stval=0x0000000000003000

```

随后，因为我们需要给缺页异常以新的处理逻辑，我们需要在用户态抛出异常，被检查到时，跳转的逻辑，这时 `scause==0xf` 就不必直接引发异常并杀死进程了。

```

void usertrap(void)
{
    ...
    else if ((which_dev = devintr()) != 0)
    {
        // ok
    }
    else if (r_scause() == 0xf) // 添加有关写缺页的异常处理
    {
        uint64 vmaddr = r_stval();
    }
}

```

```

        if (vmaddr >= p->sz) // 超限检查, 如果这个引发缺
// 异常的虚拟地址超出了进程空间大小
        {
            p->killed = 1;
        }
        else
        {
            if (lazy_handler(vmaddr, p) < 0) // 这里就是
// 真正要进行的懒分配处理器
            {
                p->killed = 1;
            }
        }
    }
    else
    ...
}

```

接下来, 在该函数之前, 我们要完成 `lazy_handler` 的定义, 这里我们其实可以直接仿照 `uvmmalloc` 的逻辑, 只不过我们在这时只需要处理单页的缺页问题, 因此有很多内容与参数可以省去。

```

int lazy_handler(uint64 va, struct proc *p)
{
    char *mem;
    // 这里的a就不用像uvmmalloc中那样作为一页一页遍历的指针
    // 了
    uint64 a = PGROUNDDOWN(va);

    mem = kalloc();
    if (mem == NULL)
    {
        // 如果物理内存真没了, 直接报错就行
        // 不同于uvmmalloc中, 如果分配到这一页物理内存用完, 要
        // 把之前的全部解映射
        // 这里什么都还没映射, 不需要 dealloc
        return -1;
    }
}

```

```

memset(mem, 0, PGSIZE);

// 第 1 次映射：映射到用户页表
if (mappages(p->pagetable, a, PGSIZE, (uint64)mem, PTE_W | PTE_X | PTE_R | PTE_U) != 0)
{
    kfree(mem); // 映射失败，只需把刚拿到的这一页物理内存还回去
    return -1;
}

// 第 2 次映射：映射到内核页表
if (mappages(p->kpagetable, a, PGSIZE, (uint64)mem, PTE_W | PTE_X | PTE_R) != 0)
{
    // 内核映射失败，必须撤销刚才成功的用户态映射
    // 注意：只 unmap 这一页 (a)，数量是 1，最后一个参数 1 表示连带 kfree 释放底层物理内存
    vmunmap(p->pagetable, a, 1, 1);
    return -1;
}

return 0;
}

```

此外，要记得在开头添加这几个声明，否则我们在新函数中的调用是无法被正确编译的：

```

#include "include/kalloc.h"
#include "include/vm.h"
#include "include/string.h"

```

这样这个节点就顺利通过！

▼ mem_cow

- 对于cow的过程，在我的**课程笔记/寻址逻辑**中有一些思考。
- 在part5-mem下创建新分支 `git checkout -b part5-cow` 。

- 仍旧先实现 `getpgcnt`，这个不再赘述。
- 接下来，看一下需要做什么，在下面的代码中标出得分规则

```
#include "test.h"

#define SIZE (1 << 12) // 4KB (one page)
#define PGSIZE (1 << 12)

/*
 * Desc:
 * We test copy-on-write by forking a process and having both parent and child
 * read from the same memory. Then we modify the memory in one process and check
 * that the physical page count increases appropriately.
 *
 * Expected:
 * After fork, physical page count should remain the same (shared pages).
 * After writing to a page in either process, the page should be copied and
 * the physical page count should increase by one.
 */

int main() {
    printf("Testing Copy-on-Write\n");

    // Get initial physical page count
    int initial_pages = getpgcnt();
    printf("Initial physical pages: %d\n", initial_pages);

    // Allocate and initialize a page
    char *mem = sbrk(SIZE);
    if (mem == (char*)-1) {
        printf("sbrk failed\n");
        exit(1);
    }
}
```

```

    }

    int write_sum = 0;

    for (int i = 0; i < SIZE; i++) {
        mem[i] = i % 256;
        write_sum += mem[i];
    }

    int after_init_pages = getpgcnt();
    printf("Physical pages after initialization: %d\n", after_init_pages);

    // 正确实现getpgcnt以后，在第一次sbrk分配一页空间并进行
    // 写操作后，
    // 现在的已分配物理页数应当比刚开始多1（因此这里不要求一定
    // 得用lazy分配才行）
    if (after_init_pages != initial_pages + 1) {
        printf("ERROR: Expected %d pages, got %d\n",
            initial_pages + 1, after_init_pages);
        exit(1);
    }

    int sz = getprocsz();
    int delta_without_cow =
        (sz / PGSIZE)    // 为子进程完全创建与父进程一样数
        + 2              // 这是给用户态的页表目录，由于RIS
        + 2              // CV在这里用的是三级页表，因此添加二级页表和三级页表目录，要单独
        + 2              // 分配两页
        + 2              // 这是给内核态的页表，这是xv6在这
        + 1              // 的变体，为了内核方便读写用户内容，当前进程用户内存也会映射到内核
        + 1              // 页表，跟上面的+2一个意义
        + 1              // 进程用户态的根页表，也就是一级
        + 2              // 页表，这个页表的基地址一般就存在SATP中了，allocproc中
        + 2              // mapping trampoline，在地址
        + 2              // 最高端，跨越了很大虚拟空间，因此不能与用户数据共用页表目录，要额
        + 2              // 外分配一级二级页表目录

```

```

        + 1 // trapframe per proc, 这一步
的额外分配页表过程可以在proc.c的allocproc找到
        + 1 // kernel page table, 内核态根
页表
        + 1 + 2 // +1: 真正的内核栈页面; +2: 对于
该内核栈的两级页表目录
    ;

    // Fork a process
    int pid = fork();
    if (pid < 0) {
        printf("fork failed\n");
        exit(1);
    }

    int after_fork_pages = getpgcnt();
    printf("Physical pages after fork: %d\n", after_f
ork_pages);

    // 如果没有实现cow, 那么这一步应该由前者等于后者, 则无法通
过测试点
    if (after_fork_pages >= after_init_pages + delta_
without_cow) {
        printf("ERROR: Page count changed too much fr
om %d to %d after fork without write\n", after_init_p
ages, after_fork_pages);
    }

    if (pid == 0) {
        // Child process
        // Read from the shared page (should not trig
ger copy)
        int child_before_read = getpgcnt();
        printf("Physical pages before child read: %d
\n", child_before_read);

        int sum = 0;
        for (int i = 0; i < SIZE; i++) {

```

```

        sum += mem[i];
    }
    printf("Child read sum: %d\n", sum);

    // 之前写进父进程页面的内容应该可以被子进程完全正确地
    读入，否则说明cow的映射在读时有问题
    if (sum != write_sum) {
        printf("ERROR: Data corruption. Sum should be %d, but got %d\n", write_sum, sum);
        exit(2);
    }

    int child_after_read = getpgcnt();
    printf("Physical pages after child read: %d (should be same)\n", child_after_read);

    // 读完页面应该不触发写权限异常，因为cow是可读不可写的权限
    if (child_after_read != child_before_read) {
        printf("ERROR: Page count changed after read\n");
        exit(3);
    }

    // Write to the page (should trigger copy)
    mem[0] = 0xFF;
    int child_write_pages = getpgcnt();
    printf("Physical pages after child write: %d (should be increased)\n", child_write_pages);

    // 写的时候应当触发越权，即缺页异常，并复制一个新页，
    再将子进程映射过去
    if (child_write_pages < after_fork_pages + 1)
    {
        printf("ERROR: Expected at least %d pages, got %d\n", after_fork_pages + 1, child_write_pages);
        exit(1);
    }

```

```

    }

    exit(0);
} else {
    // Parent process
    int status ;
    wait(&status);
    printf("wait status:%d\n", status);
    int before_write_pages = getpgcnt();

    // Check that parent's page is unchanged
    int sum = 0;
    for (int i = 0; i < SIZE; i++) {
        sum += mem[i];
    }
    printf("Parent read sum: %d\n", sum);

    // 如果正确实现子进程cow, mem[0] = 0xff不会影响到父进程的加和
    if (sum != write_sum) {
        printf("ERROR: Data corruption. Sum should be %d, but got %d\n", write_sum, sum);
        exit(1);
    }

    int after_read_pages = getpgcnt();
    printf("Physical pages after child exit: %d (should be same as before read)\n", after_read_pages);

    // 父进程读那些cow页面也不应该触发异常, 因此前后物理页面数应当不变
    if (after_read_pages != before_write_pages) {
        printf("ERROR: Expected %d pages, got %d\n", before_write_pages, after_read_pages);
        exit(1);
    }
}

```

```

        // Parent writes to the page (should not trigger copy as child is gone)
        mem[0] = 0xAA;
        int parent_write_pages = getpgcnt();
        printf("Physical pages after parent write: %d (should be same)\n", parent_write_pages);

        // 子进程已经离去，因此父进程应当不再存在cow的不可写问题，因此不会触发缺页，而是直接在现有页面写
        if (parent_write_pages != after_read_pages) {
            printf("ERROR: Page count changed after parent write\n");
            exit(1);
        }
    }

    printf("Copy-on-Write Test Completed Successfully\n");
    exit(0);
}

```

如果直接运行，在第一步就会得到这样的结果：

```

init: starting test_mem_cow
testing output size:771, contents:
Testing Copy-on-Write
Initial physical pages: 40
Physical pages after initialization: 41 (should be + 1)
Parent proc pages: 4
Physical pages after fork: 57
ERROR: Page count changed too much from 41 to 57 after fork without write
Parent proc pages: 4
Physical pages after fork: 57
ERROR: Page count changed too much from 41 to 57 after fork without write
Physical pages before child read: 57

```

```

Child read sum: 522240
Physical pages after child read: 57 (should be same)
Physical pages after child write: 57 (should be increased)
ERROR: Expected at least 58 pages, got 57
wait status:1
Parent read sum: 522240
Physical pages after child exit: 41 (should be same as before read)
Physical pages after parent write: 41 (should be same)
Copy-on-Write Test Completed Successfully
init: process pid=2 exited

```

在这里我看了一下父进程本身需要的页面数，进行计算后，子进程在cow实现前，会造成所有物理页面数增加到57页的情况，这是合理的。

- 我们需要关注 `fork` 的具体实现，可以发现最核心的复制逻辑在于 `allocproc` 这一步，顺藤摸瓜，在这个函数中，我们又找到了 `proc_pagetable`，这个函数是用来创建用户态页表的，但这个函数并不包含将用户态的内存信息copy到子进程这一步，不过会把trampoline的页面建立映射。

```

extern char trampoline[]; // trampoline.S
...

// Create a user page table for a given process,
// with no user memory, but with trampoline pages.
pagetable_t
proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;

    // An empty page table.
    pagetable = uvmcreate();
    if(pagetable == 0)
        return NULL;

    // map the trampoline code (for system call return)
    // at the highest user virtual address.

```

```

// only the supervisor uses it, on the way
// to/from user space, so not PTE_U.
if(mappages(pagetable, TRAMPOLINE, PGSIZE,
            (uint64)trampoline, PTE_R | PTE_X) < 0)
{
    uvmfree(pagetable, 0);
    return NULL;
}

// map the trapframe just below TRAMPOLINE, for tra
mpoline.S.
if(mappages(pagetable, TRAPFRAME, PGSIZE,
            (uint64)(p->trapframe), PTE_R | PTE_W)
< 0){
    vmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return NULL;
}

return pagetable;
}

```

其中，第一次建立映射是将 `trampoline.S` 的真实物理地址利用 `mappages` 挂到 `TRAMPOLINE` 定义的用户态最高地址空间处，并设置好可读可执行的权限，这里可以再看看 `mappages` 的具体实现情况

```

// Create PTEs for virtual addresses starting at va t
hat refer to
// physical addresses starting at pa. va and size mig
ht not
// be page-aligned. Returns 0 on success, -1 if walk
() couldn't
// allocate a needed page-table page.
int
mappages(pagetable_t pagetable, uint64 va, uint64 siz
e, uint64 pa, int perm)
{
    uint64 a, last;

```



```

pte_t *pte;

a = PGROUNDDOWN(va);
last = PGROUNDDOWN(va + size - 1);

for(;;){
    if((pte = walk(pagetable, a, 1)) == NULL)
        return -1;
    if(*pte & PTE_V)
        panic("remap");
    *pte = PA2PTE(pa) | perm | PTE_V;
    if(a == last)
        break;
    a += PGSIZE;
    pa += PGSIZE;
}
return 0;
}

```

综合上面的具体实例，这个函数的过程就是：将传入的 `va`（`TRAMPOLINE` 定义的用户态最高地址）和 `size` 先进行处理，给出被映射的虚拟地址起始页地址 `a` 与结束页地址 `last`；由于物理地址 `pa` 一定是页对齐的，因此无需处理；随后用 `walk` 函数，在三级页表中层层解析，找到最低一层页表给出的 `pte`，这时的 `pte` 就已经是指向物理地址的 `pte` 了；如果这个 `pte` 的 `PTE_V` 即valid位为1，则触发重分配的报错，否则将需要映射到虚拟地址的真实物理地址 `pa` 以及其它权限都放入这个 `pte`。

- 接下来我们需要调整这一套逻辑，因为我们要引入一个新的权限位，即 `PTE_COW`。选中 `PTE_V`，跳转到定义，我们在 `riscv.h` 中添加这一条定义 `#define PTE_COW (1L << 8)`，当这个位被设置时，该 `pte` 对应的物理页即为一个COW状态下的被多个进程同时共享的映射。

```

#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // 1 -> user can access
#define PTE_COW (1L << 8) // 咱们自己定义的 COW 标志位！

```

根据助教的流程，下一步我们在 `kalloc.c` 中维护一个全局的数组，用于记录每个真实物理页面被多少个进程同时映射，当只有一个计数时，该页面不再是 `cow` 的，当计数清零时，这个页面才会被 `kfree` 彻底回收，加入空闲队列。

```
struct
{
    struct spinlock lock;
    struct run *freelist;
    uint64 npage;
    uint64 ref_cnt[PHYSTOP / PGSIZE];
} kmem;
```

`PHYSTOP` 为所有物理地址的终点，而物理页要对齐 `PGSIZE`，因此这个数组的大小这样定义。事实上，我们真正可以操作的部分，即可以用于进程分配页表的部分，还是从 `kernel_end` 开始往后的物理内存空间。

接下来我们还需要维护这个 `ref_cnt`，这首先需要我们对它进行初始化（这里要对 `freerange` 进行操作），并且在 `kalloc` 时增加引用计数，在 `kfree` 时减少引用计数的同时，判断是否降为0后进行彻底回收。

```
void freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char *)PGROUNDUP((uint64)pa_start);
    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
    {
        // 注意到在freerange中最后每一个物理页都会调用一次kfree
        // 这时会自动执行ref_cnt的减少，因此我们在调用前将其设为1，这样在初始化结束后
        // 每个物理页的引用数才会是0
        acquire(&kmem.lock);
        kmem.ref_cnt[(uint64)p / PGSIZE] = 1;
        total_pages++;
        release(&kmem.lock);
        kfree(p);
    }
}
```

```

// Free the page of physical memory pointed at by v,
// which normally should have been returned by a
// call to kalloc(). (The exception is when
// initializing the allocator; see kinit above.)
void kfree(void *pa)
{
    struct run *r;

    if (((uint64)pa % PGSIZE) != 0 || (char *)pa < kern
el_end || (uint64)pa >= PHYSTOP)
        panic("kfree");
    acquire(&kmem.lock);
    if (--kmem.ref_cnt[(uint64)pa / PGSIZE] > 0)
    {
        release(&kmem.lock);
        return;
    }
    release(&kmem.lock);

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run *)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    kmem.npage++;
    release(&kmem.lock);
}

// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
    struct run *r;

```

```

    acquire(&kmem.lock);
    r = kmem.freelist;
    if (r)
    {
        kmem.freelist = r->next;
        kmem.npage--;
    }
    release(&kmem.lock);

    if (r)
    {
        memset((char *)r, 5, PGSIZE); // fill with junk
        acquire(&kmem.lock);
        kmem.ref_cnt[(uint64)r / PGSIZE] = 1;
        release(&kmem.lock);
    }
    return (void *)r;
}

```

除此之外，页面的被引用数还会在 `fork` 中被增加，因此需要补充一个函数，作为接口暴露给 `proc.c` 使用。

```

// 新增：安全地增加引用计数
void kaddref(void *pa)
{
    acquire(&kmem.lock);
    kmem.ref_cnt[(uint64)pa / PGSIZE]++;
    release(&kmem.lock);
}

```

▼ 实际上，`kernel_end` 分隔开的，低物理地址的位置存放着内核态的全部代码，所有进程的内核态都共享这部分内容，那为什么还要在 `proc` 中声明一个 `kpagetable`，在 `fork` 的时候大费周章地把所有进程都共享的内核态代码页表再复制一份出来呢？因为在这个 xv6 的实现中，为了方便该进程的内核态直接使用用户态数据，它在 `proc` 中定义的 `kpagetable` 会在用户态的 `pagetable` 映射完成后也进行一次映射，如此，虽然增加了物理内存开销，但是会节省内核直接使用用户态数据的时间。

```

// initialize kernel pagetable for each process.
pagetable_t
proc_kpagetable()
{
    pagetable_t kpt = (pagetable_t) kalloc();
    if (kpt == NULL)
        return NULL;
    // 这里的kernel_pagetable就是vm.c开头定义与初始化的内
    // 核态不变的共享代码的页表部分
    memmove(kpt, kernel_pagetable, PGSIZE);

    // remap stack and trampoline, because they shar
    // e the same page table of level 1 and 0
    char *pstack = kalloc();
    if (pstack == NULL)
        goto fail;
    if (mappages(kpt, VKSTACK, PGSIZE, (uint64)pstack, PTE_R | PTE_W) != 0)
        goto fail;

    return kpt;

fail:
    kvmfree(kpt, 1);
    return NULL;
}

```

因此，在 `kpagetable` 中，第一部分是所有进程共享的内核态代码的页表项（由 `memmove` 直接复制），第二部分才是给这个内核态开始映射一段内核栈等等。

而在 `allocproc` 时的用户态页表，创建的过程就是 `proc.c` 定义的这个函数实现的：

```

// Create a user page table for a given process,
// with no user memory, but with trampoline pages.
pagetable_t
proc_pagetable(struct proc *p)

```

```

{
    pagetable_t pagetable;

    // An empty page table.
    // 这块跟proc_kpagetable其实都是kalloc了一个新页作为
    // 页表
    pagetable = uvmcreate();
    if(pagetable == 0)
        return NULL;

    // map the trampoline code (for system call return)
    // at the highest user virtual address.
    // only the supervisor uses it, on the way
    // to/from user space, so not PTE_U.
    // 将trampoline映射到用户态内存空间的最顶端TRAMPOLINE
    // 处
    if(mappages(pagetable, TRAMPOLINE, PGSIZE,
                (uint64)trampoline, PTE_R | PTE_X) <
        0){
        uvmfree(pagetable, 0);
        return NULL;
    }

    // 为trapframe创建页表项，映射到
    // map the trapframe just below TRAMPOLINE, for
    // trampoline.S.
    if(mappages(pagetable, TRAPFRAME, PGSIZE,
                (uint64)(p->trapframe), PTE_R | PTE_
W) < 0){
        vmunmap(pagetable, TRAMPOLINE, 1, 0);
        uvmfree(pagetable, 0);
        return NULL;
    }

    return pagetable;
}

```

注意：其实 `mappages` 做的事情就是把给的物理地址与 `flags` 组合，并从第一个参数 `pagetable`（一般来说就是一级页表基址）开始，`walk` 到一个由第二个参数 `va` 解码出来的 `pte`，并修改 `pte` 的内容，这就是映射的过程。

- 不过在我们的cow中，完全没有必要对内核这部分的映射关系做调整，像助教说的，我们只需要实现对数据页面的Copy-on-Write就能节省fork时的页面复制开销，通过测试样例。
- 接下来我们对 `fork` 下手。在原版的 `fork` 中，会先调用 `allocproc`，为子进程分配一个空白的 `proc`，并且给它 `kalloc` 一个 `trapframe` 页、一个用户态页表以及一个内核态页表，这部分我们不需要改动。在接下来，进行数据页面的复制时，引发了这个函数

```
// Given a parent process's page table, copy
// its memory into a child's page table.
// Copies both the page table and the
// physical memory.
// returns 0 on success, -1 on failure.
// frees any allocated pages on failure.
int
uvmcopy(pagetable_t old, pagetable_t new, pagetable_t
knew, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i = 0, ki = 0;
    uint flags;
    char *mem;

    while (i < sz){
        if((pte = walk(old, i, 0)) == NULL)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == NULL)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) !
= 0) {
```

```

        kfree(mem);
        goto err;
    }
    i += PGSIZE;
    if(mappages(knew, ki, PGSIZE, (uint64)mem, flags
& ~PTE_U) != 0){
        goto err;
    }
    ki += PGSIZE;
}
return 0;

err:
vmunmap(knew, 0, ki / PGSIZE, 0);
vmunmap(new, 0, i / PGSIZE, 1);
return -1;
}

```

它会从旧页表基地址（父进程的用户态一级页表基地址）开始 `walk`，找到 `sz` 页数量内每一页对应 `pte`，并转化为真实的物理页地址；随后用 `mem` 进行新物理页的分配，然后 `memmove` 复制进新的页；这时 `mem` 是新分配的真实物理页的物理地址，`mappages` 会把 `mem` 映射到传入的第二个参数 `new`（子进程的用户态一级页表基地址，这个成员变量是一个 `uint64` 的指针，被定义在 `proc` 中，并被 `allocproc` 初始化好了），同时还会映射一次到 `knew`（子进程内核态一级页表基址）。

所以修改这个逻辑，就只需要让原先的映射逻辑由“将新的物理页映射到子进程页表”改为“将父进程的物理页映射过去”，同时，记得进行 `PTE_W` 与 `PTE_COW` 的设置。

```

int uvmcopy(pagetable_t old, pagetable_t new, pagetable_t knew, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i = 0, ki = 0;
    uint flags;

    while (i < sz)
    {

```



```

    if ((pte = walk(old, i, 0)) == NULL)
        panic("uvmcopy: pte should exist");
    if ((*pte & PTE_V) == 0)
        panic("uvmcopy: page not present");
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    if (flags & PTE_W)
    {
        flags &= ~PTE_W; // remove write permission
        flags |= PTE_COW; // set COW flag
        *pte |= flags;    // update parent's PTE
    }
    if (mappages(new, i, PGSIZE, (uint64)pa, flags) != 0)
    {
        goto err;
    }
    i += PGSIZE;
    if (mappages(knew, ki, PGSIZE, (uint64)pa, flags & ~PTE_U) != 0)
    {
        goto err;
    }
    ki += PGSIZE;
    kaddref((void *)pa); // 增加物理页的引用计数
}
return 0;

err:
    vmunmap(knew, 0, ki / PGSIZE, 0);
    vmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}

```

- 接下来我们进行 `trap.c` 的处理，在现在的cow机制中，如果我们对页面进行读是没问题，但是写的时候会触发缺页异常，这时候需要我们把 `cow_handler` 的相关内容实现

```

void usertrap(void)
{
    ...
    else if ((which_dev = devintr()) != 0)
    {
        // ok
    }
    else if (r_scause() == 15)
    {
        if (cow_handler(p, r_stval()) != 0)
        {
            p->killed = 1;
        }
    }
    else
    {
        ...
    }
}

```

- 接下来我们看看cow_handler应该怎么实现

```

int cow_handler(struct proc *p, uint64 va)
{
    // 首先检查引发缺页异常的页是否合法
    va = PGROUNDDOWN(va);
    if (va >= p->sz)
        return -1;

    // 找到引发缺页的对应pte并检查权限
    pte_t *pte = walk(p->pagetable, va, 0);
    if (pte == 0)
        return -1;
    if ((*pte & PTE_V) == 0)
        return -1;
    if ((*pte & PTE_U) == 0)
        return -1;
    // 这说明并非由COW引起，那就是纯纯的越权
    if ((*pte & PTE_COW) == 0)
        return -1;
}

```

```

uint64 pa = PTE2PA(*pte);
uint flags = PTE_FLAGS(*pte);

// 这一步是检查，如果子进程都结束了对该页面的cow映射，那该
// 页面被引用是1
// 此时应该完成该进程内核态和用户态页面PTE_W的恢复工作，
// 否则在最后一个测试点不能通过
uint64 ref_cnt = kgetref((void *)pa);
if (ref_cnt == 1)
{
    flags = (flags & ~PTE_COW) | PTE_W;
    *pte = PA2PTE(pa) | flags;

    pte_t *kpte = walk(p->kpagetable, va, 0);
    if (kpte)
    {
        *kpte = PA2PTE(pa) | (flags & ~PTE_U);
    }
    sfence_vma();
    return 0;
}

// 不然就是真的需要复制到新的页面
char *mem = kalloc();
if (mem == 0)
    return -1;
// 进行内容的复制，将父进程物理页的内容完全复制到新开辟的m
// em中
memmove(mem, (char *)pa, PGSIZE);
// 恢复页面的PTE_W权限，不必使用mappages重新建立映射，直
// 接修改pte的权限就行
flags = (flags & ~PTE_COW) | PTE_W;
*pte = PA2PTE(mem) | flags;

pte_t *kpte = walk(p->kpagetable, va, 0);
if (kpte)
{

```

```

    *kpte = PA2PTE(mem) | (flags & ~PTE_U);
}

// 这里其实并没有把父进程的那个页面直接释放掉，如果还有别的
// 进程仍然在cow映射
// 则这个调用只能起到减少ref的作用
kfree((void *)pa);

sfence_vma();

return 0;
}

```

其实这个调用中，很多的函数如 `walk` 等等，如果我们直接放在 `trap.c` 里，是需要补充一大堆引用的，所以我们直接放在 `vm.c` 里即可，并且记住在 `vm.h` 中添加一个函数声明。

- 最后大功告成，运行 `make run_test`，可以看到现在 `fork` 以后，比实现cow之前少了4页，而这4页就是我们之前得到的父进程的size，也就是说我们的cow确实将子进程的那些页面直接映射到父进程那些数据页面的物理页上，而不是分配一个新的物理页了。